

Java Programming

Understand polymorphism and modern Java

TAP2

Java review
Inheritance, polymorphism, generics
Functional style and modern Java

Java Review for Advanced Programming Techniques

```
public interface ReportGenerator {  
    Report generate(Input input);  
}  
// Advanced Java = designing contracts  
// that can evolve, be tested, and be reused.
```

Interfaces · Abstract Classes · Generics · Functional Java · Modern Java

Why review Java?

Students have seen Java before, but advanced topics depend on a precise understanding of typing, polymorphism, interfaces, generics, lambdas, records and the JVM.

Learning goals

A bridge from introductory Java to advanced software design

Understand contracts

Use interfaces and abstract classes intentionally, not mechanically.

Control variation

Recognize polymorphism, dynamic binding, overriding and composition.

Use generic code

Write type-safe reusable code with generics, bounds and wildcards.

Think functionally

Use lambdas, streams and immutable data where they simplify design.

Read modern Java

Know records, sealed classes, switch expressions, pattern matching and virtual threads.

Prepare for AI-assisted work

Write tests and specs that let humans and agents modify code safely.

The TAP Java baseline

Using the course GitHub examples as a living codebase

TAP repository

- TAP1 Java fundamentals and language concepts
- TAP2 OOP, inheritance, interfaces, generics
- TAP3/TAP4 Advanced techniques
- TapGroovy Dynamic JVM language examples
- TAPScalaF Functional programming on the JVM

We will reuse the old examples, but update the explanations and connect them to modern Java and AI-assisted engineering.

Modernize old Java 1.4/5 examples
Introduce small refactorings instead of isolated snippets
Prepare students for frameworks, testing and spec-driven assignments

1 • Java mental model

Primitive values vs object references

Many Java bugs are misunderstanding this slide

```
int a = 3;
int b = a;
b = 7;      // a is still 3
Point p1 = new Point(0, 0);
Point p2 = p1;
p2.setLocation(5, 5);
// p1 and p2 refer to the same object
```

Key idea

Java passes arguments by value. For objects, the value being copied is the reference, not the object itself.

Typical exam trap

"modifyPoint(p, i)" can mutate the Point object through its reference, but cannot change the caller's int variable.

Design consequence

Prefer immutable objects when shared references would make reasoning difficult.

Static vs instance members

A class-level variable is shared by all instances

```
class Account {  
    private String owner;  
    private static double interestRate = 0.03;  
    Account(String owner) {  
        this.owner = owner;  
    }  
    static void setInterestRate(double rate) {  
        interestRate = rate;  
        // owner = "Ana"; // ERROR: no instance here  
    }  
}
```

Account.setInterestRate(0.04);

Static method: called on the class

Instance method: called on one object

Static fields are global state inside a class

Be careful: static state complicates tests

Question for students: why is Math.sqrt(9) static, but account.withdraw(20) is not?

Identity, equality and modern records

`equals()`, `hashCode()` and `toString()` are not decorative

```
class Car {
    private final String name;
    private final int price;
    @Override
    public boolean equals(Object obj) {
        if (!(obj instanceof Car other)) return false;
        return price == other.price
            && name.equals(other.name);
    }
}
record Car(String name, int price) { }
// record automatically provides constructor,
// accessors, equals, hashCode and toString.
```

`==`

Compares primitive values, or whether two references point to the same object.

`equals()`

Defines domain equality. Required when objects are stored in collections.

`record`

Best for immutable data carriers. Less boilerplate, more explicit intent.

2 · Inheritance and polymorphism

Polymorphism in Java: three mechanisms

The same word describes different binding decisions

```
// 1) Subtype polymorphism: runtime dispatch
Shape s = new Circle();
s.area();           // Circle.area()
// 2) Parametric polymorphism: generics
List<String> names = new ArrayList<>();
names.add("Ada");
// 3) Overload polymorphism: compile-time choice
log("Ada");
log(42);
```

Subtype

One interface or superclass reference can point to many concrete implementations.

Parametric

One class or method works for many types through type parameters such as T.

Overload

Several methods share a name; the compiler selects by argument types.

Subtype polymorphism

Dynamic dispatch: the object decides the implementation at runtime

```
interface Shape {  
    double area();  
}  
class Circle implements Shape {  
    public double area() { return Math.PI * r * r; }  
}  
class Rectangle implements Shape {  
    public double area() { return width * height; }  
}  
Shape s = new Circle();  
System.out.println(s.area()); // Circle.area()
```

Where it appears

Interfaces, abstract classes, inheritance and overridden methods.

When binding happens

The variable type is Shape, but the invoked method is chosen at runtime.

Design value

Client code depends on a stable contract, not on concrete classes.

Parametric polymorphism

Generics: write one algorithm while preserving type information

```
class Box<T> {  
    private T value;  
    Box(T value) { this.value = value; }  
    T get() { return value; }  
}  
static <T> T first(List<T> xs) {  
    return xs.get(0);  
}  
Box<Integer> score = new Box<>(10);  
String name = first(List.of("Ada", "Linus"));
```

Where it appears

Generic classes, methods, collections, Optional<T> and Stream<T>.

When binding happens

The compiler checks type consistency; Java erases most type parameters at runtime.

Design value

Reuse data structures and algorithms without losing compile-time safety.

Overload polymorphism

Ad-hoc polymorphism: same method name, different signatures

```
class Printer {  
    void print(String text) {  
        System.out.println(text);  
    }  
    void print(int value) {  
        System.out.println(value);  
    }  
    void print(Object value) {  
        System.out.println(value);  
    }  
}  
Printer p = new Printer();  
p.print("Ada"); // print(String)
```

Where it appears

Constructors, utility APIs and methods that accept several parameter types.

When binding happens

The compiler selects the most specific compatible signature before runtime.

Do not confuse with overriding

Overloading changes parameters; overriding changes implementation in a subtype.

Composition vs inheritance

"HAS-A" and "IS-A" are design decisions

```
// Composition: a Report has many Sections
class Report {
    private final List<Section> sections;
}
// Inheritance: PdfReportGenerator is a ReportGenerator
interface ReportGenerator {
    Report generate(Input input);
}
class PdfReportGenerator implements ReportGenerator {
    public Report generate(Input input) { ... }
}
```

Use composition when

You want to build an object from stable parts, and behaviour should be delegated.

Use inheritance when

A subtype can safely substitute the parent in every context.

Warning

Inheritance is a coupling mechanism, not just a reuse mechanism.

Overriding and dynamic binding

The static type restricts access; the dynamic type chooses the implementation

```
abstract class Animal {
    abstract String speak();
}
class Dog extends Animal {
    @Override String speak() { return "woof"; }
    void fetch() { }
}
Animal a = new Dog();
System.out.println(a.speak()); // "woof"
// a.fetch();                  // compile error
if (a instanceof Dog d) {
    d.fetch();                  // safe narrowing
}
Static type: what the compiler lets you call
Dynamic type: the real class of the object at runtime
Overriding: same signature, new implementation
Dynamic binding: Java dispatches to the dynamic type
```

Advanced idea: polymorphism lets us add new behaviours without modifying client code.

Abstract classes

Partial implementation + controlled extension

```
abstract class ReportExporter {
    public final void export(Report r, Path path) {
        validate(r);
        write(r, path);
    }
    protected void validate(Report r) {
        if (r.sections().isEmpty())
            throw new IllegalArgumentException("empty report");
    }
    protected abstract void write(Report r, Path path);
}
class JsonExporter extends ReportExporter {
    @Override protected void write(Report r, Path path) { ... }
}
```

Use abstract class when

You need shared state, constructors or a template method with reusable implementation.

Do not use it when

You only need a role/contract. Then prefer an interface.

Design risk

Every protected method is an extension point you must maintain.

Interfaces

Contracts, roles and multiple views of the same object

```
interface Speaker {  
    String speak();  
}  
interface Trainable {  
    void train(Command command);  
}  
class Dog extends Animal implements Speaker, Trainable {  
    public String speak() { return "woof"; }  
    public void train(Command command) { ... }  
}  
Speaker s = new Dog();  
Trainable t = new Dog();
```

Multiple inheritance?

Java does not allow extending multiple classes, but a class can implement multiple interfaces.

Why it matters

Interfaces let us write code against capabilities instead of concrete classes.

Framework connection

Most frameworks discover and inject implementations of interfaces.

Default methods and interface evolution

Adding behaviour without breaking all implementers

```
interface Shape {
    double area();
    default String label() {
        return "area=" + area();
    }
}
class Circle implements Shape {
    private final double radius;
    public double area() {
        return Math.PI * radius * radius;
    }
}
System.out.println(new Circle(2).label());
```

Default methods help evolve APIs
They can reduce boilerplate
They can create conflicts when several interfaces define the same default method
They are not a replacement for class inheritance

Exam question: when two interfaces provide the same default method, what must the class do?

The diamond problem in Java

Why multiple class inheritance is forbidden

```
interface A {
    default String name() { return "A"; }
}
interface B {
    default String name() { return "B"; }
}
class C implements A, B {
    @Override
    public String name() {
        return A.super.name() + "/" + B.super.name();
    }
}
```

Problem

If two parents provide the same method, which one should the child inherit?

Java answer

Only one superclass. Multiple interfaces are allowed, but conflicts must be resolved explicitly.

Design lesson

Prefer small capability interfaces. Avoid "fat" interfaces that become inheritance trees in disguise.

3 • Generics and collections

Generics: from runtime errors to compile-time guarantees

The compiler becomes part of the design

```
// Before generics
List farm = new ArrayList();
farm.add(new Dog());
farm.add("not an animal");
Animal a = (Animal) farm.get(1); // runtime error
// With generics
List<Animal> farm = new ArrayList<>();
farm.add(new Dog());
farm.add("not an animal"); // compile error
Animal a = farm.get(0);      // no cast
```

What generics buy

Better documentation, fewer casts, earlier errors, safer refactoring.

What they do not buy

They do not create a new runtime class for each type parameter.

Modern syntax

Use the diamond operator: `new ArrayList<>()` when the compiler can infer the type.

Generic classes and methods

Parametric polymorphism: algorithms independent of one concrete type

```
final class Box<T> {  
    private final T value;  
    Box(T value) { this.value = value; }  
    T value() { return value; }  
}  
static <T> List<T> copyOf(Iterable<T> source) {  
    List<T> result = new ArrayList<>();  
    for (T item : source) result.add(item);  
    return result;  
}  
Box<String> b = new Box<>("TAP");  
List<Integer> xs = copyOf(List.of(1, 2, 3));
```

T is a placeholder chosen by the caller

The algorithm can be reused for many types
The compiler tracks the relationship among input and output types
Generic methods are often better than generic classes when state is not needed

Bounds and wildcards

When "any subtype" and "exactly this type" are not the same

```
class Shelter<T extends Animal> {  
    private final List<T> animals = new ArrayList<>();  
    void add(T animal) { animals.add(animal); }  
    void makeNoise() { animals.forEach(Animal::speak); }  
}  
static void printAnimals(List<? extends Animal> animals) {  
    for (Animal a : animals) System.out.println(a.speak());  
    // animals.add(new Dog()); // not allowed  
}  
static void addDogs(List<? super Dog> animals) {  
    animals.add(new Dog());  
}
```

extends

Producer: you can read Animal values safely. You usually cannot add.

super

Consumer: you can add Dog values. Reading gives Object unless narrowed.

PECS

Producer Extends, Consumer Super.

Type erasure

Generics are mostly a compile-time feature

```
List<String> names = new ArrayList<>();
List<Integer> ages = new ArrayList<>();
System.out.println(names.getClass() == ages.getClass());
// true: both are ArrayList at runtime
List raw = names;
raw.add(42);           // unchecked warning
String s = names.get(0); // ClassCastException later
```

Why erasure exists

Compatibility with older Java bytecode and libraries.

Practical consequences

No new T(), no new T[], no instanceof List<String>, and raw types are dangerous.

Modern rule

Avoid raw types. Treat unchecked warnings as design smells.

Collections: choose by behaviour

List, Set and Map are interfaces, not implementation details

```
List<String> ordered = new ArrayList<>();
Set<String> unique = new HashSet<>();
Map<String, Student> byId = new HashMap<>();
for (String name : ordered) {
    System.out.println(name);
}
byId.forEach((id, student) ->
    System.out.println(id + ": " + student.name()));
```

List

Ordered sequence. Duplicates allowed. Index-based access.

Set

No duplicates. Membership and uniqueness are the main semantics.

Map

Key-value association. Often the best replacement for parallel arrays.

Design rule: expose interfaces in APIs, choose implementations internally.

4 • Functional Java

From Comparator classes to lambdas

Less ceremony, same type system

```
record Person(String name, int age) { }  
List<Person> people = List.of(  
    new Person("Ana", 20),  
    new Person("Pedro", 24),  
    new Person("Lisa", 18)  
);  
people.sort(Comparator.comparing(Person::name));  
people.sort(Comparator.comparingInt(Person::age));  
people.forEach(p -> System.out.println(p.name()));
```

Functional interface

An interface with exactly one abstract method. A lambda can implement it.

Method reference

Person::name means: call name() on the Person parameter.

Course link

This is the modern version of old Comparator examples.

From custom map to Java streams

Old TAP idea, modern Java implementation

```
// Old idea: pass a function into an algorithm
interface Mapper<T> {
    T apply(T elem);
}
// Modern Java: use java.util.function.Function
List<String> words = List.of("java", "tap", "lambda");
List<String> upper = words.stream()
    .map(String::toUpperCase)
    .filter(w -> w.length() > 3)
    .toList();
```

Streams process data declaratively

map transforms elements
filter selects elements
collect/toList produces a result
Avoid side effects inside stream operations

Functional style is not only syntax

It is a discipline for reducing accidental state

```
// Imperative
List<String> result = new ArrayList<>();
for (User u : users) {
    if (u.active()) {
        result.add(u.name().toUpperCase());
    }
}

// Functional
List<String> result = users.stream()
    .filter(User::active)
    .map(User::name)
    .map(String::toUpperCase)
    .toList();
```

Benefits

More local reasoning, fewer temporary variables, better composition.

Costs

Can become unreadable if the pipeline hides complex domain logic.

Rule

Use functional style when it clarifies the data transformation.

5 · Modern Java after Java 8

Major Java additions in recent years

From "verbose enterprise Java" to a more expressive, concurrent language

```
virtual threads
record patterns
pattern switch
sequenced collections
// Modern Java is less about boilerplate
record Point(int x, int y) { }
String label(Object o) {
    return switch (o) {
        case Point(int x, int y) -> "Point " + x + "," + y;
        case String s             -> s.toUpperCase();
        default                   -> "unknown";
    };
}
```

Java 8

Lambdas
Streams
Optional

Java 10

var for
local variables

Java 14-16

switch expressions
records
pattern instanceof

Java 17 LTS

sealed classes
strong encapsulation

ADDITIONAL REVIEW

Small language features that still appear in real code and exams

```
List<String> names = List.of("Ana", "Bob");  
for (String name : names) {  
    System.out.println(name.toUpperCase());  
}
```

Missing Java 5+ essentials from TAP4

Why add these slides?

Students often know the syntax, but not the design reason: type safety, iteration protocols, enums as types, and convenience features.

From TAP4

Java 5 introduced generics, enhanced for, autoboxing, enums, varargs and static imports. These changed everyday Java style.

Connection to advanced topics

These features prepare students for collections, functional interfaces, streams, APIs and framework code.

LANGUAGE REVIEW

Not only "old syntax": these features are still the basis of modern Java APIs

```
// Before Java 5: raw types + casts
List farm = new ArrayList();
farm.add(perro);
Animal a = (Animal) farm.get(0);
// Java 5+: generic, safer, clearer
List<Animal> farm = new ArrayList<>();
farm.add(perro);
Animal a = farm.get(0);
```

Java 5 changed everyday Java

Java 5 changed everyday Java

Generics

List<Animal> instead of raw List
Why: compile-time type safety

Enhanced for

for (Animal a : farm)
Why: iteration over Iterable<T>

Autoboxing

list.add(5)
Why: bridge primitives and wrappers

ITERATION

Use it when you need to visit elements, not control the iterator itself

```
// Old style: explicit Iterator
Iterator<Animal> it = farm.iterator();
while (it.hasNext()) {
    Animal a = it.next();
    a.talk();
}
// Enhanced for: clearer intention
for (Animal a : farm) {
    a.talk();
}
```

Enhanced for: the iterator pattern hidden in syntax

Before

Before

After

Works with arrays

for (int x : numbers) { ... }

Works with collections

ITERATION

The language syntax depends on a small interface contract

```
public interface Iterable<T> {
    Iterator<T> iterator();
}
public interface Iterator<E> {
    boolean hasNext();
    E next();
}
public final class Farm implements Iterable<Animal> {
    private final List<Animal> animals = new ArrayList<>();
    public void add(Animal animal) {
        animals.add(animal);
    }
    @Override
    public Iterator<Animal> iterator() {
        return animals.iterator();
    }
}
Farm farm = new Farm();
farm.add(new Dog());
farm.add(new Cat());
for (Animal animal : farm) {
    System.out.println(animal.talk());
}
```

Iterable<T>: make your own objects usable in for-each

The protocol

A custom iterable class

Client code

Advanced idea

Iterable separates data structure representation from traversal. This is a classic design principle: expose behaviour, hide implementation.

TYPES

Convenient, but important to understand when primitives become objects

```
// Before Java 5
Vector<Integer> values = new Vector<>();
values.add(new Integer(5));
int n = values.lastElement().intValue();
// Java 5+
List<Integer> values = new ArrayList<>();
values.add(5);           // int -> Integer
int n = values.get(0);   // Integer -> int
Integer maybe = null;
// int x = maybe; // NullPointerException at runtime
long sum = 0;
for (Integer v : values) {
    sum += v;           // repeated unboxing
}
```

Autoboxing and unboxing

Explicit boxing/unboxing

Automatic conversion

Why it exists

Collections store objects, not primitive values. Autoboxing makes generic collections easier to use.

Common trap

Null unboxing throws `NullPointerException`.

ENUMS

Enums model closed sets of values: states, formats, roles, commands, seasons...

```
// Weak design: int constants
public static final int PDF = 0;
public static final int JSON = 1;
public static final int CSV = 2;
int format = 42; // compiles, but meaningless
// Stronger design: enum type
public enum ReportFormat {
    PDF, JSON, CSV
}
ReportFormat format = ReportFormat.JSON;
switch (format) {
    case PDF -> exportPdf(report);
    case JSON -> exportJson(report);
    case CSV -> exportCsv(report);
}
```

Enums: replace magic constants with real types

Enums: replace magic constants with real types

Before

Before

After

Compile-time safety

Invalid values do not compile.

Invalid values do not compile.

ENUMS

They can have fields, constructors and methods

```
public enum Coin {
    PENNY(1), NICKEL(5), DIME(10), QUARTER(25);
    private final int value;
    Coin(int value) {
        this.value = value;
    }
    public int value() {
        return value;
    }
}
Coin c = Coin.DIME;
System.out.println(c);           // DIME
System.out.println(c.value());   // 10
Coin parsed = Coin.valueOf("PENNY");
for (Coin coin : Coin.values()) {
    System.out.println(coin + ": " + coin.value());
}
```

Enums are classes

Enum with state and behaviour

Using enum methods

Important detail

Each enum value is a singleton instance of the enum class. Comparing with == is valid for enum values.

Design use

Prefer enums over strings when the set of legal values is closed and known.

ENUMS

Useful for small strategy-like decisions, but avoid overloading enums with too much logic

```
public enum ReportFormat {  
    PDF {  
        @Override String extension() { return ".pdf"; }  
    },  
    JSON {  
        @Override String extension() { return ".json"; }  
    },  
    CSV {  
        @Override String extension() { return ".csv"; }  
    };  
    abstract String extension();  
}  
ReportFormat f = ReportFormat.JSON;  
Path output = Path.of("report" + f.extension());  
// Good for small domain rules.  
// For large behaviour, prefer Strategy classes.
```

Enums can also hold behaviour

Constant-specific methods

Using it

Connection to TAP

This is a bridge between Java syntax and design patterns: sometimes an enum is enough; sometimes we need polymorphic classes.

CONVENIENCE FEATURES

Convenient for APIs such as logging, validation and builders

```
public static void log(String level, String... messages) {
    for (String msg : messages) {
        System.out.println "[" + level + " ] " + msg);
    }
}
log("INFO", "starting");
log("WARN", "missing config", "using default");
public static int sum(int... values) {
    int total = 0;
    for (int v : values) total += v;
    return total;
}
```

Varargs: variable number of arguments

Syntax: Type... name

Rule

Only the last parameter can be varargs.

Runtime idea

A varargs parameter is treated as an array inside the method.

Warning

Be careful with overloaded methods: varargs can make method resolution less obvious.

CONVENIENCE FEATURES

Shorten access to static constants and methods - useful, but easy to abuse

```
// Without static import
double r = Math.sqrt(Math.pow(x, 2) + Math.pow(y, 2));
System.out.println(r);
// With static import
import static java.lang.Math.*;
import static java.lang.System.out;
double r = sqrt(pow(x, 2) + pow(y, 2));
out.println(r);
import static org.junit.jupiter.api.Assertions.*;
@Test
void priceCannotBeNegative() {
    assertThrows(IllegalArgumentException.class, () -> new
Price(-1));
}
```

Static imports

Normal style

Static import style

Static import style

Good use

Mathematical formulas, constants, tests and DSL-like APIs.

Bad use

When readers cannot tell where a method or constant comes from.

SUMMARY

These details are small, but they appear everywhere in modern Java code

Checklist: Java features students should now recognize

Enhanced for

Use over arrays and `Iterable<T>` objects

Iterable/Iterator

Understand the protocol behind iteration syntax

Autoboxing

Know when primitives become wrapper objects

Enums

Closed sets of typed values; enums are classes

Varargs

Cleaner APIs for variable-length parameters

Static imports

Reduce noise, especially in tests and math-heavy code

Connection to the next part of the course: these features support clean APIs, collections, lambdas, testing frameworks, and AI-assisted code review.

Records and sealed classes

Algebraic-data-type thinking enters mainstream Java

```
sealed interface Shape permits Circle, Rectangle { }
record Circle(double radius) implements Shape { }
record Rectangle(double width, double height) implements
Shape { }
double area(Shape s) {
    return switch (s) {
        case Circle(double r) -> Math.PI * r * r;
        case Rectangle(double w, double h) -> w * h;
    };
}
```

record

Transparent immutable data carrier with generated constructor/accessors/equality.

sealed

Restricts which classes can implement or extend a type.

pattern switch

Lets the compiler check exhaustiveness for sealed hierarchies.

Virtual threads

A major platform change for scalable server-side Java

```
try (var executor =  
    Executors.newVirtualThreadPerTaskExecutor()) {  
    for (URI uri : uris) {  
        executor.submit(() -> fetch(uri));  
    }  
}  
// Keep simple blocking code,  
// but run many concurrent tasks efficiently.
```

What changed

The JVM can schedule many lightweight virtual threads over fewer platform threads.

Why students care

Concurrency no longer always means callbacks, reactive chains or manual thread pools.

Not magic

CPU-bound work, shared mutable state and synchronization problems still require design.

Compact source files and beginner-friendly Java

Java is also changing how students write first programs

```
// Newer Java aims to reduce ceremony for small programs
void main() {
    IO.println("Hello, TAP");
}
// But advanced programming still requires:
// packages, modules, visibility, tests, APIs,
// classes, interfaces and architecture.
Good for scripts and early exercises
Avoids forcing public static void main too early
Does not replace object-oriented design
Useful to compare "programming in the small" vs "programming in the
large"
```

Pedagogical opportunity: start simple, then expand the same code into professional structure.

6 · Java for AI-assisted software engineering

Why Java fundamentals matter for AI-assisted coding

AI can generate code; you must control the design

```
// Spec: add a Markdown exporter without changing PDF
export.
interface ReportExporter {
    void export(Report report, OutputStream out);
}
final class PdfExporter implements ReportExporter { ... }
final class MarkdownExporter implements ReportExporter {
    ... }
// Test protects existing behaviour
@Test void pdfExporterStillWorks() { ... }
```

Interfaces

Give the AI an explicit extension point.

Generics

Keep generated code type-safe.

Tests

Reject plausible but wrong implementations.

Specs

Define what must change and what must not break.

Spec-driven Java exercise

A practical assignment format for this course

```
change-add-markdown-export/
```

```
├─ proposal.md
```

```
├─ design.md
```

```
├─ tasks.md
```

```
└─ specs/
```

```
    └─ report-export.md
```

```
# Requirement
```

```
The system shall export reports as Markdown.
```

```
The new exporter shall implement ReportExporter.
```

```
Existing PDF and JSON exporters must not change.
```

```
All exporters must have unit tests.
```

Write the specification before implementation

Use AI only after the expected behaviour is explicit

Review generated code against interfaces, tests and invariants

Commit the spec, implementation and tests together

Advanced programming = controlled change.

Advanced programming = controlled change.

Agentic programming: what changes?

From "ask for code" to "coordinate a development loop"

Developer

→ writes spec and constraints

Agent

→ edits code and proposes tests

Build system

→ compiles, runs tests, reports failures

Developer

→ reviews design and accepts/rejects changes

Good agent task

Small, testable, bounded by an interface and a spec.

Bad agent task

"Improve the project" with no acceptance criteria.

Java review connection

Strong types, contracts and tests reduce the search space for the agent.

Review checklist for students

Before moving to advanced topics

Can you explain it?

reference vs object · == vs equals · static vs instance ·
pass-by-value

Can you design it?

composition vs inheritance · abstract class vs interface ·
polymorphism

Can you type it?

generic classes · generic methods · bounds · wildcards ·
erasure

Can you modernize it?

records · sealed classes · lambdas · streams · pattern
matching

Can you control AI changes?

specs · tests · small tasks · code review · acceptance
criteria

Suggested lab sequence

Use TAP1/TAP2 code as the starting point

Lab 1 • Fix and modernize old collection examples
Lab 2 • Replace raw types with generics
Lab 3 • Refactor Animal examples with interfaces
Lab 4 • Add tests for polymorphic behaviour
Lab 5 • Convert data classes to records
Lab 6 • Add a spec-driven feature with AI assistance
Lab 7 • Review the generated code and justify design decisions

Deliverable: code + tests + short design note + AI/spec usage report.

Each lab changes an existing codebase

Students must explain which design forces motivated the change
AI is permitted only when the spec and tests are explicit
Evaluation rewards reasoning, not only compiling code

References and source material

For the instructor version

Recommended update strategy:

1. Keep the familiar TAP examples.
2. Replace screenshots with editable code snippets.
3. Add small refactoring tasks.
4. Connect each Java concept to tests, specs and AI-assisted change.

Course code repository: github.com/pedrotgn/TAP

Original TAP Java slides: OOP, Java fundamentals, generics and lambdas
Oracle Java Language Changes by Release
Oracle JDK 25 significant changes
Oracle Java 24 release announcement